

Instituto Tecnológico de Costa Rica
Escuela de Computación
IC-5701 Compiladores e Intérpretes GR 60

Documentación Proyecto 3

Profesor:

Allan Rodríguez Dávila

Trabajo elaborado por:

Brasly Villarebia Morales - 2023105915
Sebastián Rodríguez Sánchez - 2023074446

01 de febrero del 2026

Índice

Manual de usuario	3
1.1 Requisitos del sistema	3
1.2 Estructura del proyecto	3
1.3 Compilación	4
1.4 Flujo de ejecución	4
1.5 Explicación de uso	5
Pruebas de funcionalidad	5
Descripción del problema	5
Diseño del programa	6
Librerías usadas	8
Análisis de resultados	9
Bitácora	9

Manual de usuario

Este manual describe los pasos necesarios para compilar, ejecutar y utilizar el sistema desarrollado, el cual implementa un analizador léxico y sintáctico para un lenguaje imperativo orientado a la configuración de chips, utilizando JFlex y Java CUP. Además de la ampliación que permite detectar errores semánticos (compatibilidad entre tipos), y la generación de un archivo MIPS.

1.1 Requisitos del sistema

- **Sistema operativo:** Windows, Linux o macOS.
- **Lenguaje:** Java JDK 8 o superior.
- **Herramientas:**
 - JFlex 1.9.1
 - Java CUP
 - Git (para la bitácora)

1.2 Estructura del proyecto

PP2_SebastianRodriguez_BraslyVillarebia.zip

- info.txt
- documentacion/
 - Documentacion_Externa3.pdf
- Programa/
 - librerias/
 - java-cup-11b-runtime.jar
 - java-cup-11b.jar
 - jflex-full-1.9.1.jar
 - Scripts/
 - Lexer.class — clase compilada del analizador léxico
 - Lexer.flex — archivo fuente del analizador léxico (JFlex)
 - Lexer.java — código generado por JFlex
 - Lexer.java~ — código generado por JFlex
 - Main.class — clase principal del sistema que coordina la ejecución del análisis léxico y sintáctico.
 - Main.java — clase principal del sistema que coordina la ejecución del análisis léxico y sintáctico.
 - Main\$1.class — clase interna generada por el compilador de Java.
 - MipsGenerator.class — es el resultado de ejecutar “*javac -cp ".;./Librerias/*" *.java*”, es necesario para correr el programa.
 - MibsGenerator.java — contiene toda la lógica que traduce del AST y de la tabla de símbolos, a código MIPS, genera el código “.asm”.
 - Nodo.class — implementación de la estructura del árbol sintáctico (AST).
 - Nodo.java — implementación de la estructura del árbol sintáctico (AST).
 - Parser.class — código generado y compilado del analizador sintáctico.

- Parser.cup — definición de la gramática del lenguaje en Java CUP y control de errores semánticos (se decidió integrar a esta clase).
- Parser.java — código generado y compilado del analizador sintáctico.
- Parser\$CUP\$Parser\$actions.class —
- pruebas.txt — archivo de entrada utilizados para pruebas del lenguaje.
- pruebas2.txt — archivo de entrada utilizados para pruebas del lenguaje.
- sym.class — definición y compilación de los símbolos (tokens) utilizados por el parser.
- sym.java — definición y compilación de los símbolos (tokens) utilizados por el parser.
- TablaSimbolos.class — implementación de la tabla de símbolos con manejo de scopes.
- TablaSimbolos.java — implementación de la tabla de símbolos con manejo de scopes.
- TablaSimbolos\$SymbolInfo.class — clase interna que representa la información de cada símbolo.
- TablaSimbolos\$SymbolTable.class — clase interna que representa cada tabla de símbolos por scope.
- tokens.txt — archivo de salida generado por el análisis léxico.

1.3 Compilación

(más detalle en ejecución)

1.3.1 Generación del Analizador Léxico, Desde la carpeta Scripts, ejecutar: *java -jar ../Librerias/jflex-full-1.9.1.jar Lexer.flex*. Es vital ya que genera el Lexer.java (lo que reconoce los tokens del lenguaje).

1.3.2 Generación del Analizador Sintáctico, Para este punto se usa el comando *java -jar ../Librerias/java-cup-11b.jar -parser Parser Parser.cup*. El cual genera el Parser.java y el sym.java (anteriormente explicados). Ahora con la integración del detector de errores semánticos este paso es importante, ya que sino no funcionará.

1.3.3 Compilación del proyecto, Para poder compilarlo se usa *javac -cp "..." *java* (para Linux/macOS) y para Windows *javac -cp "..." *java*.

1.3.4 Ejecución del programa, Se ejecuta la prueba con: *java -cp "..." Main prueba.txt*. Este paso se logra completar al probar el archivo de prueba a elección, el cual debe de contener una arquitectura aceptada por el lenguaje. Por lo que dependiendo del nombre del archivo de prueba, se reemplaza por “pruebas.txt” en el comando recién mencionado. Esto genera tanto el árbol sintáctico y tabla de símbolos en terminal, como el analizador sintáctico en el archivo “tokens.txt”, y por último, genera el archivo.asm para MIPS, en “generado.asm”.

1.4 Flujo de ejecución

1. Se lee el archivo fuente (pruebas.txt)
2. Se ejecuta el analizador léxico.
3. Se ejecuta el analizador sintáctico.
4. Se construye:
 - Tabla de símbolos con manejo de scopes
 - Árbol sintáctico
5. Se genera el código MIPS, en el archivo generado.asm

Se reportan errores léxicos, sintácticos y semánticos.

Se imprimen las tablas de símbolos, el árbol sintáctico, y elementos del generador de MIPS.

1.5 Explicación de uso

Compilación del proyecto

- Se debe compilar el proyecto en Java, asegurándose de incluir los archivos generados por JFlex (Lexer.java) y CUP (Parser.java). Previamente explicados los comandos necesarios para llevar a cabo la compilación.
- El archivo principal Main.java coordina la ejecución del análisis léxico, sintáctico y semántico, además de invocar al generador de código MIPS. En esta clase se seleccionan los procedimientos a realizar.

Ejecución del sistema

- Se debe ejecutar el programa indicando el archivo fuente a analizar, para efectos de entrega se usan las pruebas.txt.
- El sistema procesa el archivo en tres fases: análisis léxico, análisis sintáctico y análisis semántico, este último encargado de validar la compatibilidad entre tipos y parámetros de entrada en funciones.
- Si el archivo cumple con la gramática y las reglas semánticas, se genera un archivo de salida con extensión .asm que contiene el código MIPS correspondiente.

Salida del sistema

- En consola se muestran los errores léxicos, sintácticos o semánticos encontrados, en terminal (consola).
- Se genera un archivo generado.asm con el código MIPS correspondiente.
- El archivo .asm puede ser cargado y ejecutado en QtSpim para verificar la traducción y ejecución del programa (anteriormente explicado).

Pruebas de funcionalidad

En esta sección se ejemplifica el uso del programa, mostrando casos de prueba que validan la arquitectura del lenguaje diseñado.

En la primera imagen se presenta el contenido de un archivo de prueba enviado a través del main. Este archivo contiene, de manera intencional, un error semántico para demostrar la capacidad del compilador de detectar y reportar errores. Al ejecutar el ejemplo, se observa en consola el mensaje correspondiente, confirmando el correcto funcionamiento del sistema de detección de errores semánticos.

```
1  world int gCounter = 0 endl
2  world boolean gFlag = false endl
3
4  gift int sumaPot ¿ int a, int b ? ;
5  |   local boolean r endl
6  |   r = a + b endl
7  |   return r endl
8  |   !
9
10
11 gift boolean validar ¿ int x ? ;
12 |   local boolean ok endl
```

Imagen - Ejemplo Error

```
Generación MIPS completada.
• braslyvm@VICTUS-HP:~/Escritorio/CompiladoresJFlex/Programa/Scripts$ java -cp "../Librerias/*" Main Pruebas2.txt
Error semántico en la línea 6: 'r' es de tipo boolean y se le asignó un int
Error Semantico en la línea 7: la funcion retorna un tipo int y esta retornando un boolean
Scope: Funcion sumaPot
  a : int (parametro)
  b : int (parametro)
  r : boolean (local)
```

Imagen - Error en Consola

Posteriormente, se muestra un ejemplo válido para comprobar el funcionamiento del generador de código MIPS. En este caso, el programa se ejecuta sin errores y se procede con la generación del archivo .asm, listo para ser probado en el simulador QtSpim.

```
Programa > Scripts > Pruebas2.txt
1
2
3  gift int sumaPot ¿ int a, int b ? ;
4    local int r endl
5    r = a + b endl
6    return r endl
7  !
8
9
10 coal navidad ¿ ? ;
11   local int a endl
12   local int b endl
13   local int res endl
14   local boolean estado endl
15
16   a = 54 endl
17   b = 456 endl
18
19   res = sumaPot ¿ a, b ? endl
20   show res endl
21
22   return res endl
23 !
```

Imagen - Ejemplo Válido

```
• braslyvm@VICTUS-HP:~/Escritorio/CompiladoresJFlex/Programa/Scripts$ java -cp "../Librerias/*" Main Pruebas2.txt
Scope: Funcion sumaPot
  a : int (parametro)
  b : int (parametro)
  r : int (local)
Scope: Main
  a : int (local)
  b : int (local)
  res : int (local)
  estado : boolean (local)
Scope: GLOBAL
  sumaPot : int (funcion)
inicio generacion MIPS
entrando a main
Generación MIPS completada.
• braslyvm@VICTUS-HP:~/Escritorio/CompiladoresJFlex/Programa/Scripts$
```

Imagen - Válido en Consola

La siguiente imagen evidencia el archivo generado, con toda la estructura requerida por el simulador. Tanto la sección del .data, con la declaración de variables y sus tipos, como la sección de .text, con el manejo de funciones, manejo del stack y la función principal main (etiquetada como navidad).

```
Programa > Scripts > [M] generado.asm
1  .data
2  nl: .asciiz "\n"
3
4  .text
5  .globl main
6  main:
7      jal NAVIDAD
8      li $v0, 10
9      syscall
10
11 sumaPot:
12     # Prologo de funcion
13     addiu $sp, $sp, -8
14     sw $ra, 4($sp)
15     sw $fp, 0($sp)
16     move $fp, $sp
17     # reserva local r
18     addiu $sp, $sp, -4
19     sw $zero, -8($fp)
20     lw $t2, 8($fp)
21     addiu $sp, $sp, -4
22     sw $t2, 0($sp)
23     lw $t2, 12($fp)
24     move $t9, $t2
25     lw $t8, 0($sp)
26     addiu $sp, $sp, 4
27     add $t2, $t8, $t9
28     sw $t2, -8($fp)
29     lw $v0, -8($fp)
30     j sumaPot_end_0
31 sumaPot_end_0:
32     move $sp, $fp
33     lw $fp, 0($sp)
34     lw $ra, 4($sp)
35     addiu $sp, $sp, 8
36     jr $ra
37
38 NAVIDAD:
39     # Prologo de NAVIDAD
40     addiu $sp, $sp, -8
41     sw $ra, 4($sp)
42     sw $fp, 0($sp)
43     move $fp, $sp
44     # reserva local a
45     addiu $sp, $sp, -4
46     sw $zero, -8($fp)
47     # reserva local b
```

Imagen - Archivo Generado MIPS

Finalmente, se presenta la ejecución en QtSpim, donde el resultado obtenido es 510, coincidiendo con lo esperado. Este ejemplo demuestra que el código generado sigue los lineamientos del lenguaje y cumple con lo solicitado, validando la funcionalidad del compilador.

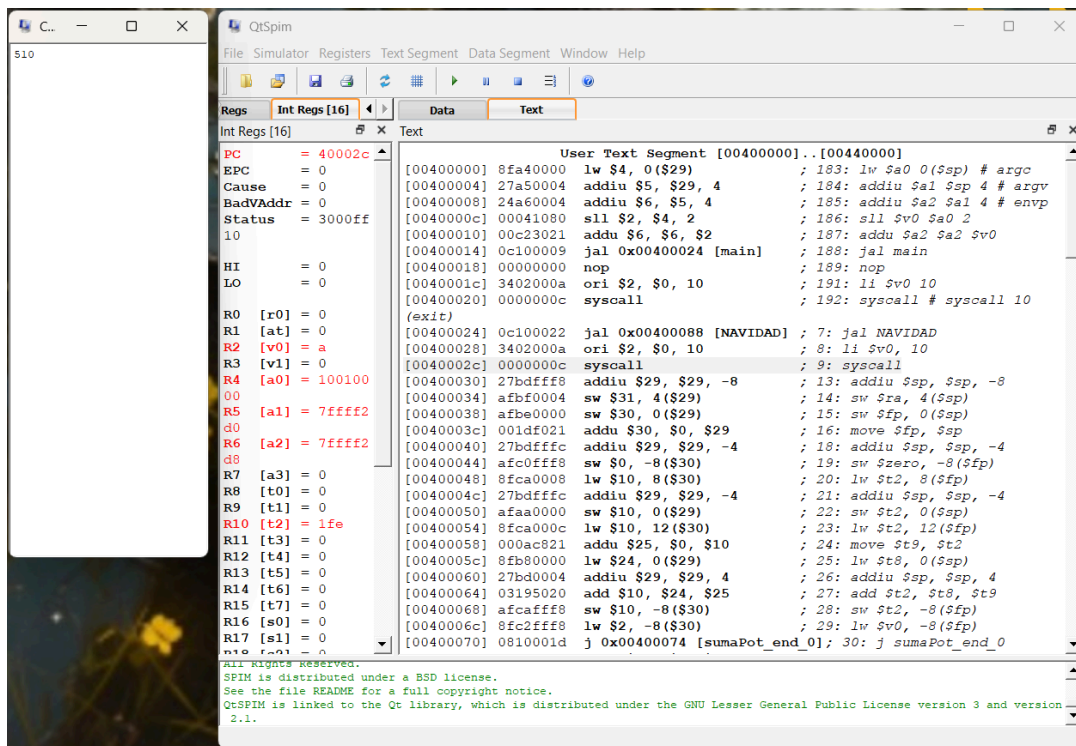


Imagen - qtSpim

Descripción del problema

Este tercer entregable corresponde al Análisis Semántico y Generación de Código Destino (MIPS). El objetivo es implementar un analizador semántico y un generador de código MIPS utilizando Java CUP y JFlex, basados en el analizador léxico y sintáctico desarrollados en los proyectos anteriores. El sistema debe validar si un archivo fuente:

- Puede ser generado por la gramática definida, y si no, reportarlo.
- Cumple con las reglas sintácticas y semánticas del lenguaje.
- Maneja correctamente errores léxicos, sintácticos y semánticos, aplicando recuperación en modo pánico (que se tiene en el cup).
- Construye el árbol sintáctico y gestiona las tablas de símbolos con scopes.
- Genera un archivo de salida con el código MIPS equivalente al archivo de prueba, funcional para QtSpim.

Diseño del programa

Se usó un diseño modular, en el que se separaron funcionalidades en clases auxiliares, para poder controlar mejor la lógica, y liberar código del cup.

Lexer.flex

El analizador léxico fue desarrollado con **JFlex** y es responsable de transformar el archivo fuente en una secuencia de tokens.

Define expresiones regulares, entre las cuales están:

- **Palabras reservadas del lenguaje**, como world, local, gift, navidad, decide, loop, for, return, show, get, entre otras.

- **Operadores aritméticos**, incluyendo binarios (+, -, *, /, //, %, ^) y unarios (++, --).
- **Operadores relacionales y lógicos**, tales como >, <, >=, <=, ==, !=, @, ~ y Σ.
- **Delimitadores y símbolos especiales**, como bloques (i, !), paréntesis (i, ?), arreglos ([,]), coma, flecha (->) y la palabra endl.
- **Literales** de tipo entero, flotante, booleano, carácter y cadena.
- **Identificadores válidos**, permitiendo letras, números y guiones bajos.
- **Comentarios de una línea** (//) y **comentarios multilínea** (/* ... */), los cuales son ignorados durante el análisis.

Se utilizan las directivas %line y %column para poder controlar la posición de cada token (información vista en clase).

Además, implementa **manejo de errores léxicos** mediante la regla comodín [^], que reporta el carácter inválido junto con su línea y columna, permitiendo continuar el análisis (recuperación en modo pánico).

Parser.cup

El analizador sintáctico fue implementado usando CUP.

Entre sus responsabilidades principales se incluyen:

- Definición de terminales y no terminales, alineados directamente con los tokens generados por el lexer.
- Establecimiento de precedencias y asociatividad para operadores aritméticos, relacionales y lógicos, garantizando la correcta evaluación de expresiones complejas.
- Implementación completa de la gramática del lenguaje, incluyendo:
 - Declaraciones de variables globales y locales.
 - Definición de funciones y del procedimiento principal navidad.
 - Manejo de estructuras de control (decide, loop, for).
 - Sentencias de asignación, retorno, lectura y escritura.
 - Manejo de arreglos bidimensionales y su inicialización.
- Construcción del árbol sintáctico, utilizando la clase Nodo.
Cada producción gramatical crea nodos jerárquicos que representan la estructura del programa fuente.
- Gestión de tablas de símbolos con manejo de scopes, coordinada directamente desde el parser:
 - Apertura de nuevos ámbitos al entrar en funciones, bloques y el método principal.
 - Cierre de ámbitos al finalizar cada bloque, imprimiendo su contenido.
 - Registro de símbolos con información de tipo, categoría y posición.
- Integración de acciones semánticas básicas, tales como:
 - Declaración de identificadores.
 - Validación del uso de símbolos previamente declarados.
 - Asociación de tipos a nodos del árbol sintáctico.
- Manejo y recuperación de errores sintácticos en modo pánico:
 - Conteo de errores sintácticos detectados durante el análisis.
 - Reporte de errores indicando línea y columna.
 - Uso de producciones con error y sincronización mediante endl para continuar el análisis.
- Validación final del archivo fuente, determinando si puede o no ser generado por la gramática:

- Si se detectan errores sintácticos, se notifica al usuario.
- Solo en ausencia de errores se imprime la tabla de símbolos global y el árbol sintáctico.

Analizador semántico

El análisis semántico se ejecuta dentro del cup apoyándose de la tabla de signos y el árbol sintáctico, se implementaron las siguientes validaciones.

- Declaración y redeclaración de identificadores:
 - Detecta variables o funciones redeclaradas en el mismo scope.
 - Controla que el identificador exista antes de ser usado.
- Verificación de tipos (tipado fuerte):
 - Expresiones aritméticas (por ejemplo, int con float si tu lenguaje lo permite, o restringido si no)
 - Comparaciones relacionales (retorno booleano, operandos válidos)
 - Operadores lógicos (operadores booleanos)
- Validación de llamadas a funciones:
 - Verifica existencia de la función.
 - Verifica cantidad de parámetros.
 - Verifica compatibilidad de tipos por posición
- Reglas semánticas de control de flujo:
 - Condiciones en decide/loop deben ser booleanas
 - validación de retorno que coincide con la declaración de la función
- Reporte de errores semánticos:
 - Emite mensajes de error con línea y columna.
 - Permite continuar (modo pánico) para reportar varios errores en una sola ejecución.

Se implementó una tabla de símbolos con manejo de scopes, donde cada bloque, función o estructura abre una nueva “rama”. Se implementó de esta forma y no como “hashmap” ya que se concluyó que era más cómodo y se podía abarcar la totalidad de lo solicitado en las especificaciones con mayor eficiencia.

Cada símbolo almacenado contiene información como:

- Nombre del identificador.
- Tipo de dato.
- Categoría (global, local, parámetro, función).

El sistema valida:

- Errores por redeclaración dentro del mismo scope.
- Uso de identificadores no declarados, reportando errores semánticos básicos durante el análisis sintáctico.

Generador de código MIPS

El generador de código MIPS fue implementado en la clase MipsGenerator y tiene como función traducir el árbol sintáctico abstracto (AST), previamente validado en las fases léxica, sintáctica y semántica, a código ensamblador MIPS ejecutable en qtSpim.

Este proceso se realiza recorriendo el árbol de manera recursiva y generando instrucciones MIPS de acuerdo con el tipo de nodo y su información semántica.

El generador se apoya directamente en la tabla de símbolos para resolver tipos de datos, categorías de los identificadores y direcciones de memoria, permitiendo un acceso correcto a variables globales, locales y parámetros de funciones.

El código generado se divide en las secciones estándar de MIPS:

La sección .data, donde se reservan:

- Variables globales y arreglos.
- Literales de tipo string, registrados con etiquetas únicas.
- Literales float, almacenados dinámicamente
- Constantes auxiliares como el salto de línea.

La sección .text, donde se genera:

- El punto de entrada main, que invoca al procedimiento principal.
- El código de todas las funciones definidas en el programa.
- Los prólogos y epílogos necesarios para el manejo del stack.

El lenguaje define como obligatorio el procedimiento principal NAVIDAD. El generador valida su existencia y, en caso de no encontrarse, se produce un error semántico que impide la generación del código. Desde main se realiza una llamada directa a main (navidad), finalizando el programa mediante un syscall de salida compatible con qtSpim.

Para las funciones, se implementa un modelo de activación basado en el uso del stack. Cada función genera automáticamente un prólogo y un epílogo que guardan y restauran los registros \$ra y \$fp, además de reservar el espacio necesario para variables locales. Los parámetros se manejan mediante offsets positivos respecto al frame pointer, mientras que las variables locales utilizan offsets negativos, manteniendo una organización clara de la memoria.

La generación de expresiones se realiza de forma recursiva, evaluando primero los operandos y utilizando el stack para almacenar resultados intermedios. Se soportan operadores aritméticos, relacionales y lógicos, así como operadores unarios como incremento y decremento.

Las expresiones enteras y booleanas utilizan registros temporales, mientras que las expresiones de tipo float emplean los registros de punto flotante y las instrucciones correspondientes.

Las comparaciones de valores flotantes se traducen usando instrucciones de comparación y generan resultados booleanos normalizados, permitiendo su uso directo en estructuras de control como *decide*, *loop* y *for*.

El generador también implementa el manejo de arreglos bidimensionales. Para acceder a un elemento, se calcula su dirección en tiempo de ejecución a partir de los índices de fila y columna, el número de

columnas y el tamaño de palabra, generando el acceso correcto a memoria tanto para lectura como para escritura.

Las sentencias de entrada y salida (get y show) se traducen a llamadas al sistema compatibles con qtSpim. Dependiendo del tipo de dato, se utilizan las syscalls correspondientes para enteros, flotantes o cadenas. Cada operación de salida incluye automáticamente un salto de línea para mejorar la legibilidad de la ejecución.

Las estructuras de control de flujo (decide, loop, for y break) se implementan mediante el uso de etiquetas generadas dinámicamente. Estas etiquetas permiten controlar los saltos condicionales y la repetición de bloques, incluso en casos de ciclos anidados, utilizando una pila interna para el manejo correcto de break.

Main.java

Coordina la ejecución del sistema:

- Lee el archivo fuente (pruebas2.txt).
- Ejecuta el análisis léxico, generando el archivo tokens.txt con el tipo de token y su lexema.
- Ejecuta el análisis sintáctico, construyendo el árbol sintáctico y gestionando las tablas de símbolos.
- Reporta errores léxicos y sintácticos de forma clara.
- Muestra mensajes de confirmación al finalizar cada fase del análisis.

Construcción del árbol sintáctico

Se construye de forma incremental, siguiendo un modelo apegado al parser, generado por CUP, (que es LALR(1)). Cada reducción crea un nodo que se agrupa a los elementos que se reconocen en el lenguaje. Dentro del manejo en el CUP, se le asigna un literal, ya que no se le puede asignar o fijarse un tipo de dato.

Gestión de la tabla de símbolos por alcance

Se maneja de forma jerárquica, respetando los distintos alcances del programa. Cada bloque, función o estructura de control abre un nuevo scope, en el cual se registran las declaraciones locales. Las búsquedas de identificadores se realizan desde el scope actual hacia los superiores, permitiendo el acceso a variables visibles y detectando errores como redeclaraciones o usos fuera de alcance.

Interacción entre componentes

- **Lexer.flex** → **Lexer.java**: genera el analizador léxico que produce tokens.
- **Parser.cup** → **Parser.java** + **sym.java**: define la gramática y los símbolos que recibe el parser.
- **Parser.java** ↔ **TablaSimbolos.java**: gestiona la declaración y uso de símbolos por scope.
- **Nodo.java**: representa el árbol sintáctico generado.
- **Main.java**: integra todos los componentes y ejecuta el análisis completo del archivo fuente.

Librerías usadas

- **JFlex 1.9.1**
Utilizada para la generación del analizador léxico a partir del archivo Lexer.flex.
Comando de uso: `java -jar ../Librerias/jflex-full-1.9.1.jar Lexer.flex`
- **Java CUP 11b**
Utilizada para la generación del parser y de la clase sym, permitiendo la definición formal de los tokens reconocidos por el analizador léxico.
Comando de uso: `java -jar ../Librerias/java-cup-11b.jar -parser Parser Parser.cup`
- **Bibliotecas Java CUP Runtime**
Incluidas dentro del classpath para el uso de la clase `java_cup.runtime.Symbol` y la interfaz `Scanner`.
- **Java estándar (JDK 17)**
Utilizado para la compilación y ejecución del proyecto, así como para el manejo de archivos y salida estándar.

Análisis de resultados

Se solucionan los errores del proyecto 2 antes de desarrollar el proyecto y el proyecto 3 se completa satisfactoriamente completando así la tercera fase del proyecto logrando el analizador semántico y el generador de MIPS:

- Conservación y mejora en la funcionalidad de la parte 1 y 2 del proyecto.
- Validación del archivo fuente determinado si el programa es generable por la gramática.
- Implementación de manejo de errores léxicos, sintácticos y semánticos aplicando el modo pánico de continuación de revisión del código fuente.
- Implementación del análisis semántico directamente en el cup validando la compatibilidad de tipos, declaraciones válidas y usos correctos de identificadores.
- Generación de código MIPS al no haber algún tipo de error se genera automáticamente el código MIPS equivalente semánticamente al programa fuente
- Uso de GitHub como sistema de control de versiones, manteniendo una bitácora de commits clara.
- Entrega de documentación interna y externa que cumple con los requerimientos del proyecto.

Bitácora

[sebro08/CompiladoresJFlex: Proyecto de compiladores e intérpretes en verano, en JFlex y Cup](#)